

Name : Nour Ehab Samir Mohamed

Id : 20241063

Sheet (1)

Implementation Level (Header File) (stack.h)

```
#ifndef STACK_H_INCLUDED
#define STACK_H_INCLUDED

#define MAX 100

typedef char Type;

typedef struct stack{
    int top;
    Type item[MAX];
} Stack;

// Q1 Prototypes
void creatStack(Stack *s);
int isEmpty(Stack s);
int isStackfull(Stack s);
void push(Type item, Stack *s);
Type pop(Stack *s);

// Q5 Type firstelement(Stack s);

// Q6
Type lastelement(Stack s);

// Q7
void destroyStack(Stack *s);

// Q8
void copyStack(Stack *src, Stack *dest);
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

// Q9

```
int StackSize(Stack s);
```

```
#endif
```

Implementation Level (stack.c)

```
#include <stdio.h>
```

```
#include "stack.h"
```

// Q1

```
void creatStack(Stack *s) {
```

```
s->top = -1;
```

```
}
```

// Q1

```
int isStackempty(Stack s) {
```

```
return s.top == -1;
```

```
}// Q1
```

```
int isStackfull(Stack s) {
```

```
return s.top == MAX - 1;
```

```
}
```

// Q1 push

```
void push(Type item, Stack *s) {
```

```
s->item[++s->top] = item;
```

```
}
```

// Q1 pop

```
Type pop(Stack *s) {
```

```
return s->item[s->top--];
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

```
}
```

```
// Q5
```

```
Type firstelement(Stack s) {
```

```
    return s.item[0];
```

```
}
```

```
// Q6
```

```
Type lastelement(Stack s) {
```

```
    return s.item[s.top];
```

```
}
```

```
// Q7
```

```
void destroyStack(Stack *s) {
```

```
    s->top = -1;
```

```
}
```

```
// Q8
```

```
int i;
```

```
void copyStack(Stack *src, Stack *dest) {creatStack(dest);
```

```
    for (i = 0; i <= src->top; i++) {
```

```
        dest->item[i] = src->item[i];
```

```
    }
```

```
    dest->top = src->top;
```

```
}
```

```
// Q9
```

```
int StackSize(Stack s) {
```

```
    return s.top + 1;}
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

User Level(main.c)

```
#include <stdio.h>
```

```
#include "stack.h"
```

```
// Q15
```

```
void printStack(Stack s) {
```

```
    if (isStackempty(s)) {
```

```
        printf("Stack is empty.\n");
```

```
        return;
```

```
    }
```

```
    printf("Stack elements (Bottom -> Top): ");
```

```
    for (int i = 0; i <= s.top; i++) {
```

```
        printf("%c ", s.item[i]);
```

```
    }printf("\n");
```

```
}
```

```
// Q13
```

```
void copyStackUser(Stack *src, Stack *dest) {
```

```
    copyStack(src, dest);
```

```
    printf("Stack copied successfully.\n");
```

```
}
```

```
// Q16
```

```
int isBalanced(char exp[]) {
```

```
    Stack s;
```

```
    creatStack(&s);
```

```
    for (int i = 0; exp[i] != '\0'; i++) {
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

```
char ch = exp[i];

if (ch == '(' ch == '{' ch == '[') {
    if (isStackfull(s)) return 0;
    push(ch, &s);
} else if (ch == ')' ch == '}' ch == ']') {
    if (isStackempty(s)) return 0;
    Type top = pop(&s);
    if ((ch == ')' && top != '(') ||
        (ch == '}' && top != '{') ||
        (ch == ']' && top != '[')) {
        return 0;
    }
}

return isStackempty(s);
}

// Q2, Q3, Q10-Q14

int main() {
    Stack s1, s2;
    char choice;

    Type item; creatStack(&s1);

    while (1) {
        printf("\n===== STACK MENU =====\n");
        printf("(a) Read element then Push it.\n");
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

```
printf("(b) Pop element then display it.\n");

printf("(c) Display first element (Bottom) - Q10.\n");

printf("(d) Display last element (Top) - Q11.\n");

printf("(e) Display stack size - Q14.\n");

printf("(f) Copy stack (s1 -> s2) - Q13.\n");

printf("(g) Print stack (s1) - Q15.\n");

printf("(h) Destroy stack (s1) - Q12.\n");

printf("(i) Check parentheses balance - Q16.\n");

printf("(x) Exit.\n");

printf("Enter choice: ");

scanf(" %c", &choice);

switch (choice) {

case 'a':

if (isStackfull(s1)) {printf("Error: Stack Overflow. Cannot push.\n");

} else {

printf("Enter element (character): ");

scanf(" %c", &item);

push(item, &s1);

printf("Pushed: %c\n", item);

}

break;

case 'b':

if (!isStackempty(s1)) {

printf("Popped: %c\n", pop(&s1));
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

```
} else {

printf("Error: Stack Underflow. Stack is empty.\n");

}

break;

case 'c':

if (!isStackempty(s1)) {

printf("First element (Bottom): %c\n", firstelement(s1));

} else {printf("Error: Stack is empty.\n");

}

break;

case 'd':

if (!isStackempty(s1)) {

printf("Last element (Top): %c\n", lastelement(s1));

} else {

printf("Error: Stack is empty.\n");

}

break;

case 'e':

printf("Stack size = %d\n", StackSize(s1));

break;

case 'f':

copyStackUser(&s1, &s2);

printf("Original Stack (s1): "); printStack(s1);

printf("Copied Stack (s2): "); printStack(s2);
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

```
break;case 'g':  
    printStack(s1);  
  
    break;  
  
    case 'h':  
  
        destroyStack(&s1);  
  
        printf("Stack destroyed (top reset to -1).\n");  
  
        break;  
  
        case 'i': {  
  
            char exp[100];  
  
            printf("Enter expression (e.g., {[(())}): ");  
  
            scanf("%s", exp);  
  
            if (isBalanced(exp)) {  
  
                printf("Result: Parentheses are BALANCED.\n");  
  
            } else {  
  
                printf("Result: Parentheses are NOT balanced.\n");  
  
            }  
  
            break;}  
  
        }  
  
        }  
  
        }  
  
        case 'x':  
  
            printf("Exiting program.\n");  
  
            return 0;  
  
            default:
```

Name : Nour Ehab Samir Mohamed

Id : 20241063

```
printf("Invalid choice. Please enter a valid option.\n");
```

```
break;
```